Video Stream Project

Matthew Hardenburgh, Jose Ramirez, Paul Krueger

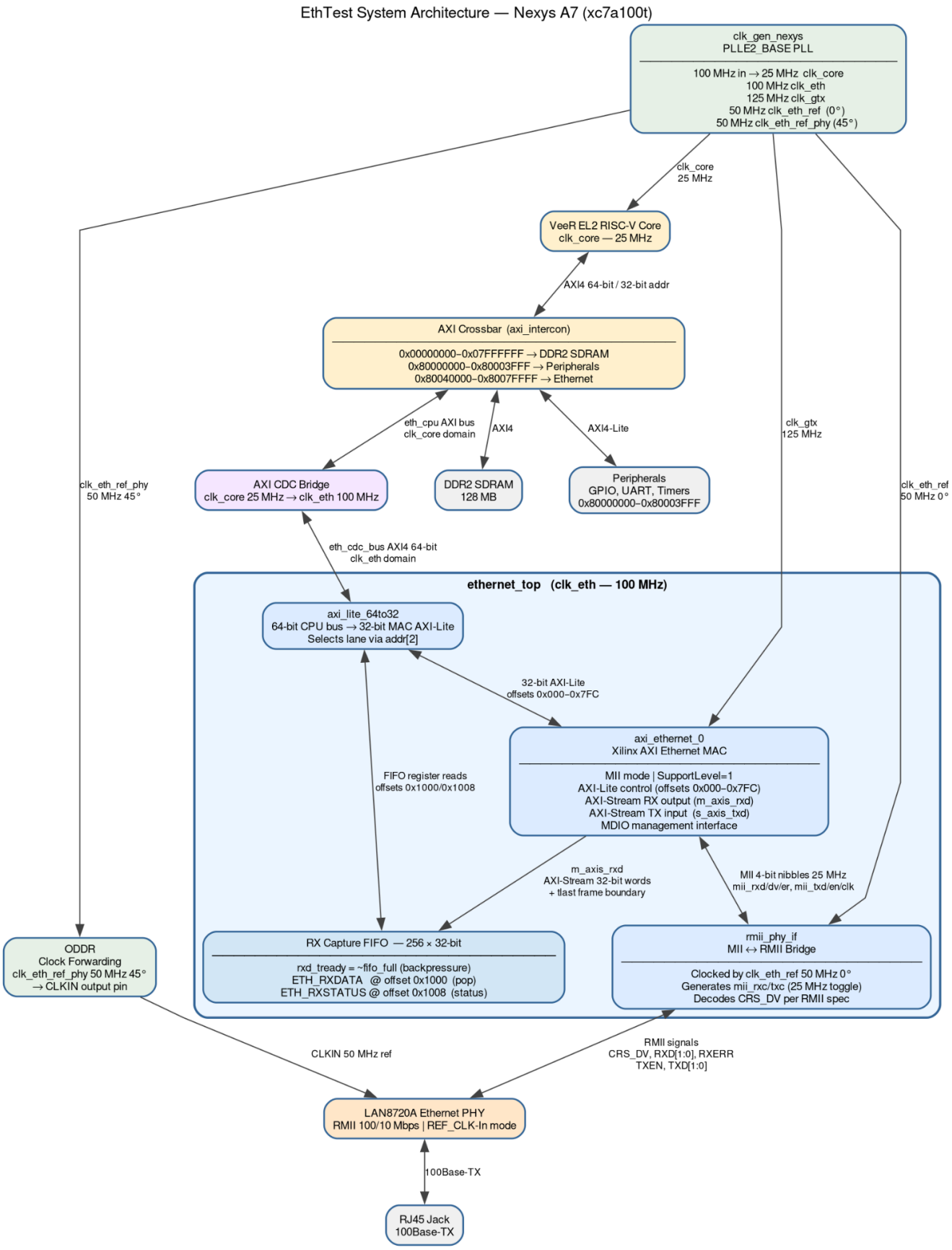
Overview

The goal of this project was straightforward, to stream video data from a desktop host PC to the Nexys 4DDR, and the Nexys 4 then displays it on VGA. The original concept was to use ethernet to accomplish this goal, as ethernet has a higher bandwidth and it came with the board. We did not want to have to purchase any additional hardware. Throughout the course of development on this project though we had encountered several challenges that caused the scope of the project to explode which required either pivots or ways to reduce the scope. To deliver a minimum viable product, we had to reduce the scope of the project from using ethernet to using UART to transfer pixel data, and from streaming video to transferring a single image frame. We also learned a significant amount on this project as well, in particular how to send and receive data in a protocol fashion.

Hardware

Ethernet

Our initial hardware design was focused on integrating the Ethernet port of the Nexys 4 board to the Project 2 Design with the VGA controller. The ethernet hardware uses preexisting IPs in order to convert the Ethernet port RMII to AXI-lite Slave ports. Four hardware ICs are used: MII-to-RMII for interfacing with the Ethernet port, the Xilinx The AXI Ethernet Subsystem with MII port settings, AXI-Lite 64-to-32 converter for connecting the Subsystem to the SoC's 64-bit AXI-lite bus, and RX Capture FIFO to convert the AXI-Stream for Ethernet packets to AXI-Lite registers.


EthTest System Architecture — Nexys A7 (xc7a100t)
clk_gen_nexys
PLLE2_BASE PLL
100 MHz in → 25 MHz clk_core
100 MHz clk_eth
125 MHz clk_gtx
50 MHz clk_eth_ref (0°)
50 MHz clk_eth_ref_phy (45°)
clk_core
25 MHz
VeeR EL2 RISC-V Core
clk_core — 25 MHz
AXI4 64-bit / 32-bit addr
AXI Crossbar (axi_intercon)
0x00000000–0x07FFFFFF → DDR2 SDRAM
0x80000000–0x80003FFF → Peripherals
0x80040000–0x8007FFFF → Ethernet
eth_cpu AXI bus
clk_core domain
AXI4
AXI4-Lite
clk_gtx
125 MHz
clk_eth_ref_phy
50 MHz 45°
clk_eth_ref
50 MHz 0°
AXI CDC Bridge
clk_core 25 MHz → clk_eth 100 MHz
DDR2 SDRAM
128 MB
Peripherals
GPIO, UART, Timers
0x80000000–0x80003FFF
eth_cdc_bus AXI4 64-bit
clk_eth domain
ethernet_top (clk_eth — 100 MHz)
axi_lite_64to32
64-bit CPU bus → 32-bit MAC AXI-Lite
Selects lane via addr[2]
32-bit AXI-Lite
offsets 0x000–0x7FC
axi_ethernet_0
Xilinx AXI Ethernet MAC
MII mode | SupportLevel=1
AXI-Lite control (offsets 0x000–0x7FC)
AXI-Stream RX output (m_axis_rxd)
AXI-Stream TX input (s_axis_txd)
MDIO management interface
FIFO register reads
offsets 0x1000/0x1008
m_axis_rxd
AXI-Stream 32-bit words
+ tlast frame boundary
MII 4-bit nibbles 25 MHz
mii_rxd/dv/er, mii_txd/en/clk
ODDR
Clock Forwarding
clk_eth_ref_phy 50 MHz 45°
→ CLKIN output pin
RX Capture FIFO — 256 × 32-bit
rxd_tready = ~fifo_full (backpressure)
ETH_RXDATA @ offset 0x1000 (pop)
ETH_RXSTATUS @ offset 0x1008 (status)
rmii_phy_if
MII ↔ RMII Bridge
Clocked by clk_eth_ref 50 MHz 0°
Generates mii_rxc/txc (25 MHz toggle)
Decodes CRS_DV per RMII spec
CLKIN 50 MHz ref
RMII signals
CRS_DV, RXD[1:0], RXERR
TXEN, TXD[1:0]
LAN8720A Ethernet PHY
RMII 100/10 Mbps | REF_CLK-In mode
100Base-TX
RJ45 Jack
100Base-TX

ethernet_top Register Map

All registers are 32-bit wide and accessed as unsigned int from firmware. The Ethernet peripheral base address is **0x80040000**.

Addresses 0x80040000 – 0x80040FFF (offset 0x000 – 0xFFF) are forwarded to the Xilinx AXI Ethernet MAC through the axi_lite_64to32 width converter.

Addresses 0x80041000 – 0x8007FFFF (offset 0x1000 – 0x3FFFF) are decoded by ethernet_top and service the internal RX Capture FIFO registers.

MAC Control / Status Registers (Xilinx DS759)

Register	CPU Address	Offset	Bit Field	Access	Reset Value	Description
RAF	0x80040000	0x000	[2] BcstRej	R/W	0x0000_0000	Receive Address Filter. BcstRej=0 accepts broadcast frames (default). Set bit 2 to reject broadcasts.
IS	0x8004000C	0x00C	[7] MgtRdy	RO	—	Management Ready. Set to 1 once PHY MDIO is initialised. Always 1 in steady state.

Register	CPU Address	Offset	Bit Field	Access	Reset Value	Description
			[6] RxDcmLock	RO	—	RX DCM lock. Always 1 (no DCM in MII mode).
			[4] RxMemOvr	W1C	0	RX Memory Overflow. Set if MAC's internal RX FIFO overflows. Cleared by writing 1.
			[3] RxRject	W1C	0	Frame Rejected. Set when a received frame is discarded (bad CRC, address mismatch, etc.). Write 1 to clear.
			[2] RxCmplt	W1C	0	Frame Complete. Set when a full frame has been received and transferred

Register	CPU Address	Offset	Bit Field	Access	Reset Value	Description
						to the AXI-Stream. Write 1 to clear.
RCW1	0x80040404	0x404	[28] RX_EN	R/W	0x1000_FFFF	Receiver Enable. **Must be set to 1** to enable frame reception. Write (1u << 28).
			[27] Vlan	R/W	—	VLAN frame enable. Leave 0 for standard frames.
			[26] HD	R/W	—	Half-duplex enable. Leave 0 for full-duplex.
EMMC	0x80040410	0x410	[31:30] MAC_SPEED	R/W	0x8000_0000	MAC speed. 10 = 1 Gbps (reset default — **wrong for 100M**), 01 = 100 Mbps, 00 = 10 Mbps.

Register	CPU Address	Offset	Bit Field	Access	Reset Value	Description
						Write (1u << 30) to select 100 Mbps.

RX Capture FIFO Registers (ethernet_top custom, offset ≥ 0x1000)

Reads to these offsets are intercepted by ethernet_top and never forwarded to the MAC.

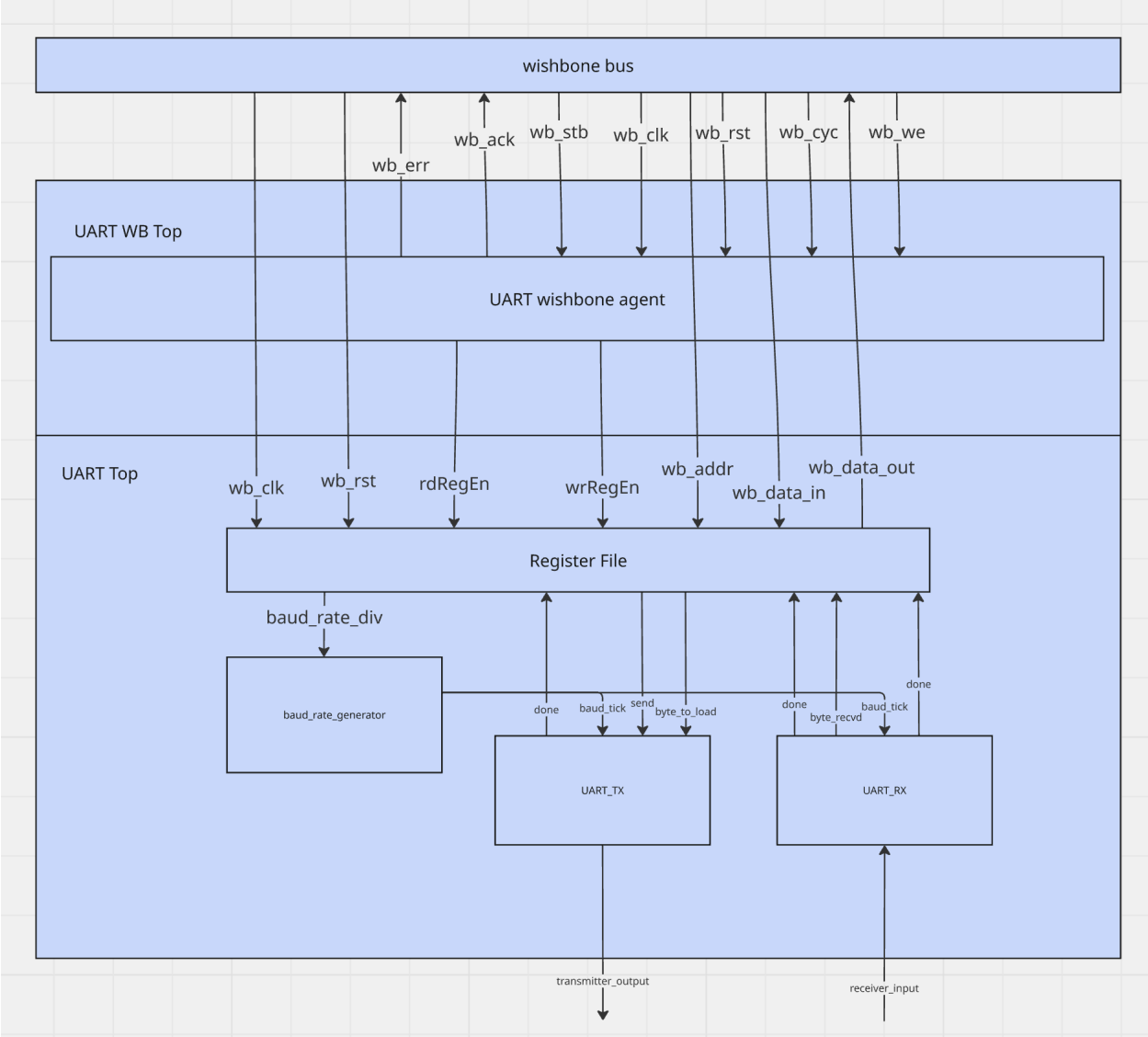
Register	CPU Address	Offset	Bit Field	Access	Reset Value	Description
ETH_RXDATA	0x80041000	0x1000	[31:0] data	R (destructive)	—	Pop one 32-bit word from the head of the RX Capture FIFO. Each read advances the read pointer by one entry. Returns 0xDEADBEEF if the FIFO is empty. Read ETH_RXSTATUS first to confirm

Register	CPU Address	Offset	Bit Field	Access	Reset Value	Description
						the FIFO is not empty.
ETH_RXSTATUS	0x80041008	0x1008	[31:24] frame_count	RO	0x0000_0001	Number of complete (tlast-terminated) frames currently in the FIFO. Increments when a frame's last word is written; decrements when that word is read out.
			[17:9] word_count	RO	—	Total 32-bit words currently stored in the FIFO (0 – 256).
			[8] tlast	RO	—	The tlast flag of the word at the current read pointer. Indicates the head word is the

Register	CPU Address	Offset	Bit Field	Access	Reset Value	Description
						last word of its frame.
			[1] full	RO	—	FIFO full flag. Set when all 256 entries are occupied. The MAC will be backpressured (rxd_tready = 0) while full; new data is held in the MAC's internal buffer.
			[0] empty	RO	1	FIFO empty flag. **Poll this bit before reading ETH_RX DATA.** 1 = no data available.

UART

For the UART peripheral used in the design pivot, we used a core that Matt developed during the summer of 2025. While the core passed all functional tests, once integrated, there were several bugs that will be elaborated on later. Below is a block diagram of the core.



Register Map

The base address is 0x80001880

Offset	Name	Type	Reset Value	Description
0x00	BAUD_RATE_GEN_REG	RW	0x00000000	Holds the number of counts until a baud clock pulse
0x04	UART_CTL_REG	RW	0x00000000	Control register, reset and tx send bit

0x08	UART_TX_STATUS_REG	RW	0x00000000	Transmit status
0x0C	UART_RX_STATUS_REG	RW	0x00000000	Receive Status
0X10	UART_TX_HOLDING_REG	RW	0x00000000	Hold byte to be transmitted
0x14	UART_RX_HOLDING_REG	RW	0x00000000	Hold byte that was received

Register Descriptions

Register 0. 0x00 BAUD_RATE_GEN_REG

Bit Field	Name	Type	Reset	Description
31:0	Baud Clock Divider	RW	0x00000000	System clock cycles per baud clock pulse

Register 1. 0x04 UART_CTL_REG

Bit Field	Name	Type	Reset	Description
0	System Reset	RW	0x0	Reset tx and rx state machines
1	Transmit Send	RW	0x0	Transmit Byte in the UART_TX_HOLDING_REG
31:2	Reserved	RO	0x0	Software should not rely on the value of a reserved bit. The value of a reserved bit should be preserved across a read-modify-write operation.

Register 2. 0x08 UART_TX_STATUS_REG

Bit Field	Name	Type	Reset	Description
0	Transmit Done	RW	0x0	New byte has been transmitted, asserted until cleared
31:1	Reserved	RO	0x0	Software should not rely on the value of a reserved bit.

Register 3. 0x0C UART_RX_STATUS_REG

Bit Field	Name	Type	Reset	Description
0	Receive Done	RW	0x0	New byte has been received, asserted until cleared
31:1	Reserved	RO	0x0	Software should not rely on the value of a reserved bit.

Register 4. 0x10 UART_TX_HOLDING_REG

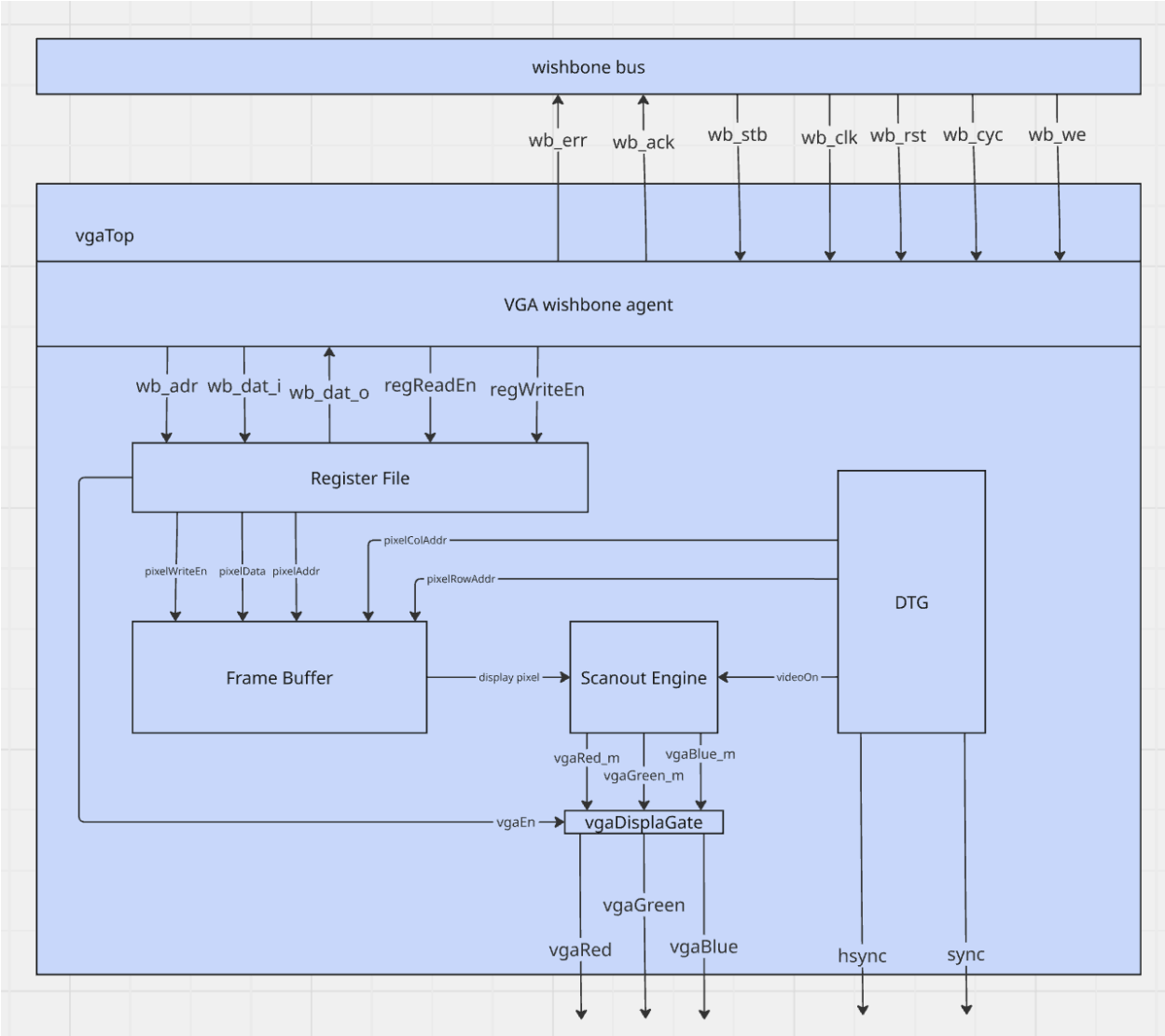
Bit Field	Name	Type	Reset	Description
7:0	TX Buffer	RW	0x0	Byte to be transmitted
31:8	Reserved	RO	0x0	Software should not rely on the value of a reserved bit. The value of a reserved bit should be preserved across a read-modify-write operation.

Register 5. 0x14 UART_RX_HOLDING_REG

Bit Field	Name	Type	Reset	Description
7:0	RX Buffer	RW	0x0	Byte received
31:8	Reserved	RO	0x0	Software should not rely on the value of a reserved bit. The value of a reserved bit should be preserved across a read-modify-write operation.

VGA

We had re-used the VGA core from Matt and Joses' project 2. It already had the desired capabilities and we knew it worked. Below is a block diagram.



Register Map

The base address is 0x80001500

Offset	Name	Type	Reset Value	Description
0x00	VGA_CTL_REG	RW	0x00000000	Main control register
0x04	VGA_OUTPUT_REG	RO	0x00000000	View output of signals. For debugging
0x08	VGA_PIXEL_ADDR_REG	RW	0x00000000	Address of pixel to be written to
0x0C	VGA_PIXEL_DATA_REG	RW	0x00000000	Data to write to pixel

Register Descriptions

Register 0. 0x00 VGA_CTL_REG

Bit Field	Name	Type	Reset	Description
0	VGA_EN	RW	0x0	enable/disable the VGA output
1	PIXEL_WR_EN	RW	0x0	enable/disable writing to the frame buffer
31:2	Reserved	RO	0x0	Software should not rely on the value of a reserved bit. The value of a reserved bit should be preserved across a read-modify-write operation.

Register 1. 0x04 VGA_OUTPUT_REG

Bit Field	Name	Type	Reset	Description
11:0	VGA_OUT	RO	0x000	Output of the vgaBlue, vgaGreen, vgaRed signals
31:12	Reserved	RO	0x00000	Software should not rely on the value of a reserved bit. The value of a reserved bit should be preserved across a read-modify-write operation.

Register 2. 0x08 VGA_PIXEL_ADDR_REG

Bit Field	Name	Type	Reset	Description
31:0	PIXEL_ADDR	RW	0x00000000	Pixel address

Register 3. 0x0C VGA_PIXEL_DATA_REG

Bit Field	Name	Type	Reset	Description
31:0	PIXEL_DATA	RW	0x00000000	Pixel data

Software

Ethernet Route

Host PC Software

The original design would be to test the functionality of the device by streaming the desktop on the host PC to the Nexys board. This would have been accomplished with a small shell script running on a Linux host that uses Pipewire to capture the desktop, pass it to ffmpeg, and then pass the packets to the ethernet port on the host using arp. To reduce overhead and complexity, raw ethernet frames would have been used, keeping the focus on the data payload and not spending bandwidth on UDP or TCP overhead.

Nexys 4 DDR Firmware

The Nexys board would have had a program running that monitored the ethernet port for packets. The packet format would have contained 1 pixel with the x and y coordinates of the video frame, and the RGB values of the pixel. It would then build up the framebuffer for the VGA peripheral to output to the monitor.

UART Route

The software used follows a rudimentary transmission protocol where a byte is sent by the host to the Nexys 4, and then the Nexys 4 acknowledges that byte. Another byte is not sent by the host until the host receives that acknowledgment. If the host does not receive an acknowledgement from the Nexys 4, then it times out.

Host PC Software

The host PC software is a python script that initiates the transfer. Python was chosen because it is multiplatform. The transfer is first initiated with a header first. Once those header bytes are acknowledged, then the pixel data transfer begins. In a loop, the pixel data for the red nibble is sent and then the software waits for an ACK, then green nibble is sent and ACKed, and finally blue nibble is sent and ACKed. The loop continues, sending all pixels. The first iteration of the host software sent red and green nibbles in one byte, followed by blue and a footer nibble in one byte. The problem with this first iteration is that when a string of 0xFFs (1s) is sent, the receiver state machine in the hardware would become desynchronized and go kaput. The second iteration

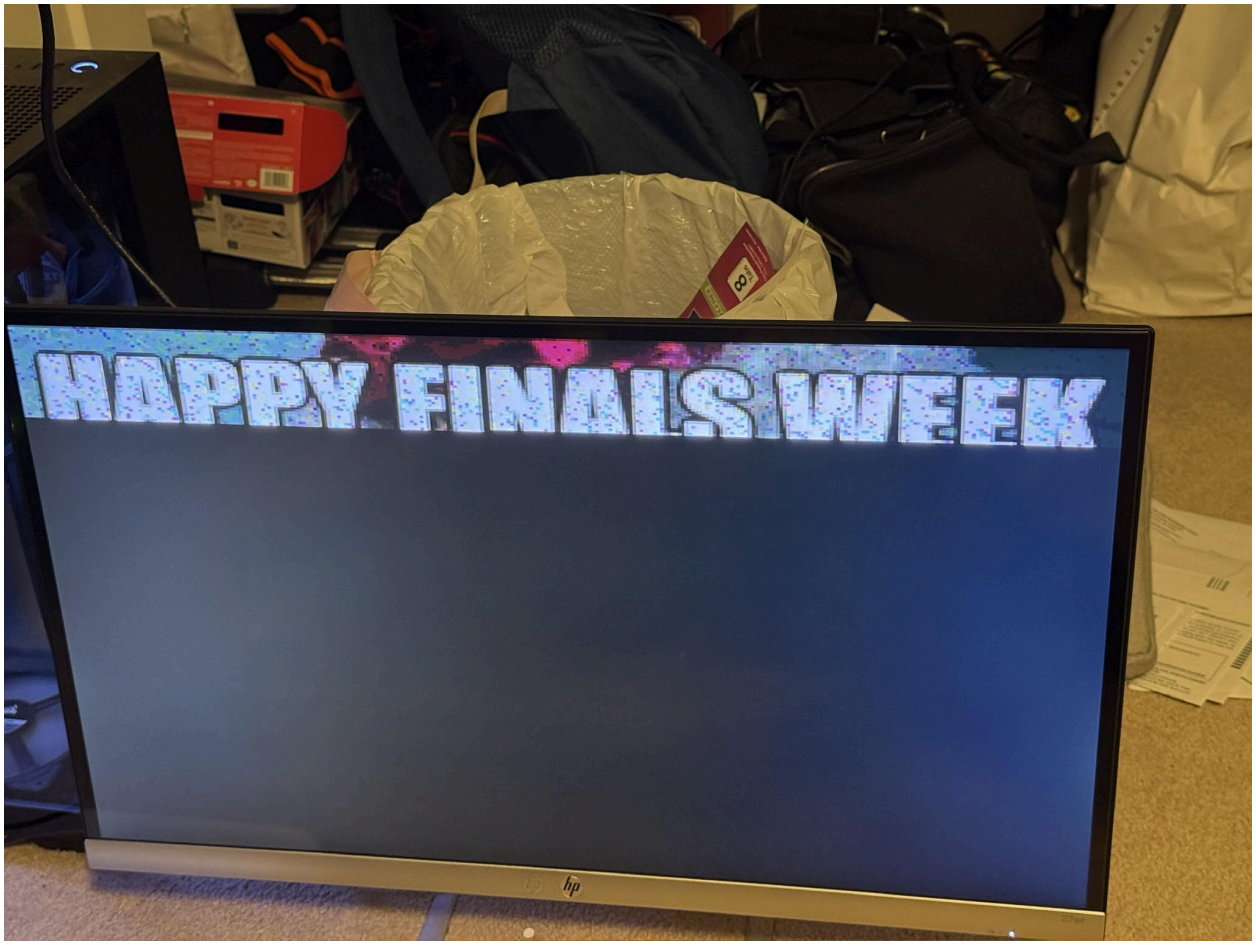
fixed this problem by sending the red, green and blue nibbles individually, padded with zeros. The host software has a generous timeout of 30 seconds to give the firmware on the Nexys to receive the pixel data, and ACK it.

Nexys 4 DDR Firmware

The firmware also operates on a loop based architecture and is the complement to the host software. After initial housekeeping of resetting the tx/rx state machines and setting the baud rate, the firmware waits for a header from the host. Once the firmware receives the header, it goes into receive and display mode. In this mode the firmware receives the red, green and blue nibbles from the host, collating them into a full pixel and writing that pixel data to the VGA controller core. When each nibble is received, an ACK is sent back to the host so that the host can send the next nibble. This repeats until all pixels are received.

Picture of Working Pixel Data Transfer

Below are pictures of a working data transfer using UART. The first picture is in the middle of the transfer, the second picture is at the end of the transfer.



Learnings and Challenges

As mentioned in the overview, the scope of this project had exploded as work started on it. One of the immediate challenges we realized was the availability of memory and how the peripherals could access that memory. When designing our own ethernet peripheral we realized memory cost was going to be an issue. The framebuffer in the VGA peripheral consumed a significant portion of the available BRAM, leaving little left for other memory intensive peripherals. The ethernet peripheral required at a minimum of 2x42 bytes (octets) for the data payloads, one set of 42 bytes for transmit and another set for receive. Proper design requires multiple payloads to be buffered to avoid missing packets, further increasing the memory cost. On top of the memory cost problem, was a memory access problem. Registers are typically 32 bits (4 bytes) and the wishbone peripheral is limited to 16 registers in total. The minimum number of registers required for the payloads are 20. Ten registers to hold the payload for transmitting, and another 10 registers to hold the payload for receiving. If we wanted to increase the ethernet data

payload to a whole 320x240 frame, there would not be enough registers to read out the entire payload. DMA would have been the solution to store the payload data in DRAM, but the EL2 core does not support DMA . Further compounding the memory cost and memory access problems was the wishbone bandwidth. At 25MHz system clock, the theoretical maximum framerate that could be transferred through wishbone is 27FPS. Since wishbone read and writes are initiated by the EL2 core, that theoretical number is degraded by the processor executing other instructions in between the register read and writes that transfers data from the ethernet core to the VGA one.

Ethernet

In the light of these issues we decided to pivot to using a Xilinx ethernet IP core that is attached to the AXI bus, and also seek out potentially using UART as a method to transfer pixel data as a back up. The AXI bus provided more bandwidth than the wishbone bus. First off, generating the bitstream with the axi_ethernet subsystem required using an evaluation license which took many attempts to correctly synthesize. There were also problems with the clock domain since the Ethernet port and MII MAC needed separate clock signals than the bus, causing timing issues. There were also issues with the AXI byte lanes where the AXI Ethernet Subsystem was only 32 bits wide while the AXI bus was 64 bits wide and it was hardware to read the half of the bus which the Ethernet register wasn't connected to. Finally, there was the issue of reading the Ethernet data packets: First we found out that the Ethernet Data Packets were being read on the AXI-Stream interface which means we needed to add a FIFO in order to add them to the AXI-Lite Register Map. Then we encountered a problem with writer pointer reset behavior as well as the reset of the whole Ethernet top module. Unfortunately, we weren’t able to read the Ethernet packet data by the end of the project which I suspect might be due to not following the high level ethernet protocol in the firmware or having the Ethernet cable connect directly between the host and the Nexys A7 board.

One difficult aspect was fully diagnosing the ethernet port's connection issues. The initial attempts at using a C program uploaded to the board led to repeated, efficient probing of different signals, which was both inefficient and ineffectual. Instead, there are many better tools available for checking if the hardware is working and where it might be failing. Vivado has a built-in logic analyzer which would have been more appropriate.

Since the initial issues revolved around the clock, making use of the logic analyzer, the MDIO signals, and perhaps even a physical oscilloscope would have pointed earlier to the clock mismatch that we were experiencing. Jumping into trying to diagnose with a C

program also skipped the hardware verification steps that should have occurred first. Ultimately, the wrong tool was chosen for what was ultimately a hardware configuration issue.

UART

As mentioned in the overview, UART was a back-up method of transferring pixel data. This was done as a way to hedge our bets on having a working minimum viable product to demonstrate. This method was not without its issues however. The IP core that was initially used had flaws with it, and a software protocol had to be developed on the host and Nexys 4 to ensure good transfers without data loss. The flaws with the IP core were discovered during the course of software development, and caused data loss, resulting in the frame from being drawn.

The first major issue was combining the bits that software writes and bits that hardware writes for control and status of the UART state machines, all into one register. In doing this, the only way to set or clear a bit, without affecting the others was to use a read-modify-write scheme; or set or clear the bit without regard for the other bits. When using the read-modify-write scheme software would miss a done bit, and that manifested as an unsent ACK on the host side, resulting in a complete stop of transfer of pixel data. This bug was fixed by separating the control bits and status bits into separate registers. Speaking of missing done bits, another bug that was found in the HDL, was that the done bits, the bits that indicate the state machine is done receiving or done transmitting, was not sticky. Sticky bits are bits that remain set until cleared. This was an issue because the design intention was that the done bits should be sticky, but the behavior was not the case. This bug also manifested as unsent ACKs, caused by the Nexys software missing the done bit, because the bit was assumed to be set and stay set when a transfer or receive was done.

The second major issue was the clock domain. The original design of the UART core was to have the transmit and receive state machines clocked by the baud rate generator. This initially seemed like a good idea, as they were operating directly on the UART signal. However, this caused cross clock domain issues as the UART registers are clocked by the system clock and the state machines are not. The UART transmit state machine would miss start[transmitting] bit set by software and then later cleared, which manifested as ACKs that were not sent to the host PC. This issue was resolved by clocking the state machines with the system clock and using the baud rate pulses to move the state machine forward. Another related issue that arose was the setting of done bits in the register

directly by the tx/rx state machine or the setting of send and reset state machine control signals directly by the register. This bug caused desynchronization that would result in missed pixels from the host to the Nexys or unACKed pixels from the Nexys to the host. Instead of a bit being set in the register directly by the tx/rx state machine, we instead use a pulse that is decoupled from the tx/rx state machine. The same was done on the register side, once the start[transmitting] bit was set, a single pulse signal would be sent to the tx state machine.

AI Usage

AI was used to assist with debugging hardware and software issues, such as pointing out typos and swapped port connections. On the UART core, AI was the one to suggest separating out the status and control registers. Working with the Xilinx licensing tools was obtuse and AI was used to figure out the process on how to use them to instantiate the ethernet IP core. For the chat logs concerning the use of AI in the project, they are attached in the project tarball in the Chat_Logs/ directory enumerated in the order they were created.

Future Improvements

Streaming the pixel data from a host PC to the Nexys 4 using ethernet would be at the top of the list for future improvements. Next would be implementing DMA so that both the VGA and Ethernet cores could use the DDR to store data. An alternative VGA implementation that did not require a full framebuffer and instead received and sent each individual pixel would have freed up BRAM for other uses. This would have come at the cost of some added complexity for the software running on the board, but would have also simplified the VGA peripheral. Also implementing a robust way to search for the device on the network would be a software improvement.

Conclusion

For this project we successfully demonstrated a minimum viable product of streaming pixel data from a host PC to the Nexys 4 to display on a monitor over the VGA connection. Unfortunately, we were unable to do this using the much faster ethernet connection, and instead had to use a UART connection. We encountered issues with BRAM memory capacity on the FPGA and bugs in the UART core. The two major bugs were combining the UART control and status registers, and working with different clock domains incorrectly. We solved these issues and were able to display an image on the monitor connected to the Nexys VGA port.